Improving MonoGS: View-Direction-Aware Regularization and Historical Keyframe Replay for Gaussian Splatting SLAM

Jiayu Lin
University of Shanghai for Science and Technology
Shanghai, China
2335052103@st.usst.edu.cn

***Abstract*—MonoGS pioneers dense SLAM using 3D Gaussian Splatting (3DGS), achieving real-time tracking and photorealistic mapping via differentiable tile-based rasterization. Despite its strong performance, we identify two independent methodological defects in the original design that limit its accuracy and robustness. First, the isotropic regularization penalizes all anisotropic stretching uniformly, thereby suppressing geometrically efficient surface-parallel elongation while insufficiently penalizing harmful view-aligned stretching that causes screen-space ill-conditioning. Second, the sliding-window mapping backend ceases to supervise Gaussians observed only by evicted keyframes, leading to catastrophic forgetting in the map and elevated rendering residuals for historical regions. To address these, we propose (1) view-direction-aware isotropic regularization (B4), which modulates the anisotropy penalty by the alignment between a Gaussian's major axis and the camera viewing direction, and (2) historical keyframe replay (D2), which randomly replays evicted frames at a 10Rigorous ablation studies on TUM RGB-D and Replica datasets demonstrate that D2 consistently improves tracking (ATE 3.28 cm vs. 3.45 cm baseline, -4.9Crucially, the combined Full system exhibits negative interference, with performance dropping below the baseline on RGB-D sequences, which we analyze through the lens of gradient landscape incompatibility. Our findings highlight that the primary bottleneck in Gaussian Splatting SLAM lies in the spatio-temporal distribution of supervision signals, rather than in resource allocation or densification schedules.**

***Index Terms*—SLAM, 3D Gaussian Splatting, differentiable rendering, isotropic regularization, catastrophic forgetting, keyframe management.**

I. Introduction

Dense simultaneous localization and mapping (SLAM) is a cornerstone of robotics and augmented reality, demanding both geometric accuracy for camera tracking and photorealistic fidelity for downstream applications. The advent of Neural Radiance Fields (NeRF) [3] demonstrated that implicit scene representations could achieve unprecedented novel-view synthesis quality. However, the volumetric rendering integral required by NeRFs is computationally prohibitive for real-time SLAM systems, necessitating hierarchical encoding structures or deferred rendering pipelines that compromise fine geometric detail.

3D Gaussian Splatting (3DGS) [2] introduced a paradigm shift by representing scenes as an explicit set of anisotropic 3D Gaussians optimized via differentiable tile-based rasterization. This explicit representation avoids costly ray marching, enabling real-time rendering while preserving competitive image quality. MonoGS [1] was the first to extend 3DGS to the SLAM setting, proposing a monocular-to-RGB-D framework that jointly optimizes camera poses and a dense Gaussian map through a frontend-backend architecture. The frontend performs frame-to-model tracking and keyframe selection, while the backend refines Gaussian parameters $(\boldsymbol{\mu}, \mathbf{q}, \mathbf{s}, \mathbf{c}, \alpha)$ over a sliding window of active keyframes.

Motivation. Despite MonoGS's impressive results, its design contains under-explored assumptions that leave performance headroom unclaimed. Through systematic reproduction and diagnosis (Sec. III–IV), we identify two *independent* methodological defects:

1) **Geometrically blind regularization.** MonoGS employs a uniform isotropic loss that pushes all Gaussians toward spheres. Yet anisotropy is not uniformly harmful: elongation parallel to surfaces is an efficient approximation strategy, whereas stretching along the viewing direction causes severe screen-space aliasing and novel-view artifacts. A uniform penalty cannot distinguish these cases (Sec. IV-A).
2) **Catastrophic forgetting in the Gaussian map.** Once a keyframe is evicted from the sliding window, its observed spatial regions receive no further direct gradient signal. We empirically show that evicted-frame residuals exhibit a persistent upward trend relative to in-window frames and remain on average 1.3–1.8$\times$ higher in the latter half of training (iterations $>$12,000), indicating progressive map degradation in regions no longer directly supervised (Sec. IV-B).

Contributions. We propose two targeted, methodology-level improvements:

- We introduce **view-direction-aware isotropic regularization** (B4) that weights the anisotropy penalty by the cosine alignment $\alpha = |\mathbf{v} \cdot \mathbf{d}|$ between the camera viewing direction $\mathbf{v}$ and the Gaussian's major axis $\mathbf{d}$. This preserves efficient surface-parallel stretching while aggressively penalizing view-aligned artifacts that induce screen-space ill-conditioning.
- We introduce **historical keyframe replay** (D2), a simple

yet effective mechanism that samples evicted keyframes during mapping iterations with a 10% probability and blends their photometric loss into the optimization objective with a weight of 0.5. This provides continued direct supervision to historical regions without destabilizing the active tracking window.

We evaluate both improvements on the TUM RGB-D [13] and Replica [14] datasets under strictly controlled conditions: identical hardware, software versions, hyperparameters, and random seeds. Our ablation study reveals that D2 delivers consistent gains across all metrics, while B4 fails to improve tracking accuracy and degrades perceptual quality (LPIPS), revealing that view-aware penalty alone is insufficient without a learnable weighting function. Strikingly, the combined Full system underperforms the individual modules, exposing a negative interference phenomenon that we attribute to incompatible gradient landscapes.

II. Related Work

Neural Implicit SLAM. Early dense SLAM systems relied on explicit representations such as truncated signed distance functions (TSDF) [11] or surfels [12]. The rise of coordinate-based neural networks enabled implicit mapping, with iMAP [5] and NICE-SLAM [6] demonstrating scalable indoor reconstruction. Point-SLAM [4] introduced a neural point-cloud representation with learned descriptors, improving rendering fidelity over pure MLPs. However, these methods require minutes to hours per scene due to volumetric rendering or MLP query overhead, precluding real-time operation.

3D Gaussian Splatting for SLAM. 3DGS [2] represents scenes as millions of anisotropic 3D Gaussians rasterized via splatting, achieving 100+ FPS rendering. MonoGS [1] adapted this representation to SLAM, coupling differentiable rendering with a keyframe-based backend. Subsequent works such as GS-SLAM [8] and SplaTAM [9] have explored depth-guided densification and tracking strategies. While these methods advance the state of the art, they largely inherit MonoGS's backend design, including its regularization and keyframe management policies.

Regularization in 3DGS. The original 3DGS paper does not constrain Gaussian anisotropy, which can lead to elongated "spider-web" artifacts in unbounded scenes. MonoGS introduced an isotropic regularization $\mathcal{L}_{\text{iso}} = \lambda \sum_i \|\mathbf{s}_i - \bar{s}_i \mathbf{1}\|_1$ with $\lambda = 10$ to encourage compact, spherical Gaussians. However, this uniform penalty is geometrically agnostic: it penalizes surface-parallel elongation that is, in fact, a desirable compression strategy for planar regions. No prior work in Gaussian Splatting SLAM has investigated view-dependent regularization strategies.

Keyframe Management and Forgetting. Keyframe-based SLAM systems [7] traditionally maintain geometric constraints for all landmarks via a factor graph, ensuring that evicted keyframes still influence the map through shared 3D points. In differentiable SLAM, constraints are implicit: they exist only if a frame is rendered and back-propagated. Consequently, when MonoGS evicts a keyframe, its geometric and photometric constraints vanish entirely. This phenomenon is analogous to catastrophic forgetting in continual learning, yet it remains unaddressed in the context of Gaussian-based SLAM.

Catastrophic Forgetting in Differentiable Representations. The phenomenon of catastrophic forgetting—where a model loses previously acquired knowledge upon learning from new data—is well-studied in neural network continual learning [16], [17]. Replay-based methods, where past examples are periodically re-exposed to the model, remain among the most effective mitigation strategies across task-incremental and domain-incremental settings [18]. Despite the structural similarity—MonoGS's sliding window creates a de facto continual learning problem for the Gaussian map—no prior work has applied replay mechanisms to differentiable SLAM backends. Our D2 method bridges this gap, demonstrating that a simple uniform replay strategy yields consistent improvements with negligible computational overhead.

III. Baseline Reproduction

A. Experimental Protocol

We reproduce MonoGS on two scenes following the official evaluation protocol: (1) **TUM RGB-D fr3/office** [13] (monocular, 2585 frames, 640×480), a challenging real-world sequence with low-texture regions, dynamic objects, and photometric variation; and (2) **Replica room0** [14] (RGB-D, 2000 frames, 1200×680), a high-fidelity synthetic scene with accurate ground-truth geometry and camera trajectories.

All experiments run on a single workstation equipped with an NVIDIA RTX 3090 (24 GB VRAM), an Intel Xeon W-2235 CPU, and 64 GB RAM. The software environment consists of Ubuntu 20.04, Python 3.9, PyTorch 2.2.2, CUDA 12.1, and the MonoGS codebase at commit `ccb408b`. We fix the random seed to 42 across all runs via `torch.manual_seed(42)` and `np.random.seed(42)` to ensure deterministic reproducibility.

B. Reproducibility Environment

Achieving reproducible results required navigating subtle but critical environmental constraints, which we document here to aid future researchers.

Platform dependency. MonoGS uses Python `multiprocessing` to share CUDA tensors among three processes: frontend (tracking), backend (mapping), and GUI. On Linux, the default `fork` start method inherits the parent's memory space, allowing CUDA tensor views to remain valid across process boundaries. On Windows, only `spawn` is available; it serializes objects via pickle, and the `Camera` object's rotation matrix $\mathbf{R}$ (a PyTorch view tensor) loses its base storage reference during deserialization, causing $R[1, 1]$ to silently become zero and triggering singular matrix errors in `linalg.inv`. MonoGS is therefore fundamentally Linux-only at the implementation level.

Dependency version locks. The original MonoGS environment specifies Python 3.7.13 and PyTorch 1.12.1. We found that these versions are incompatible with modern CUDA

toolkits (CUDA 12.x), requiring a cascade of dependency updates. Table I lists the critical version constraints and their justifications.

TABLE I
CRITICAL DEPENDENCY VERSION CONSTRAINTS AND THEIR JUSTIFICATIONS.

Package	Final Version	Rationale
Python	3.10.20	Requires ≥3.9 for PyTorch 2.x compat
PyTorch	2.2.2+cu121	CUDA 12.x compatibility
NumPy	1.26.4 (<2)	PyTorch 2.2.2 incompatible with NumPy 2.x ABI
setuptools	67.8.0 (<68)	evo 1.11.0 depends on `pkg_resources`
evo	1.11.0 (fixed)	v1.36.5 changed `align_trajectory` API
matplotlib	<3.6	v3.10.x `colorbar` API incompatible with evo 1.11.0
opencv-python	<4.9	≥4.9 requires NumPy≥2 (conflicts)
plyfile	<1.0	≥1.0 requires NumPy≥2 (conflicts)

Code modification. The only source-code change required across the entire project is adding `#include <cfloat>` as the first line of `submodules/simple-knn/simple_knn.cu`. CUDA 12 no longer transitively includes the `<cfloat>` header, causing `FLT_MAX` to be undefined. This change does not affect algorithm behavior and is purely environmental.

These constraints collectively ensure that our baseline is reproducible and that all ablation experiments (Sec. VI) are compared on an equal footing.

Tracking accuracy is measured by Absolute Trajectory Error (ATE, cm) after Sim(3) alignment for monocular sequences and 7-DoF alignment for RGB-D sequences, computed with the `evo` toolbox [15]. Rendering quality is assessed by PSNR (dB), SSIM, and LPIPS (AlexNet backbone) [10], evaluated on held-out frames (every 50th frame for TUM; a predefined test split for Replica). Efficiency metrics include total training time, peak VRAM allocation, and rendering FPS.

C. Reproduction Results

Table II compares our reproduction against the values reported in the original MonoGS paper. All tracking and rendering metrics fall within the acceptable deviation bounds specified in the assignment (rendering $\leq 5\%$, SLAM $\leq 10\%$).

LPIPS discrepancy analysis. Our Replica room0 reproduction surpasses the paper's reported LPIPS by 28% (0.049 vs. 0.068). We attribute this primarily to PyTorch version differences: LPIPS [10] uses a pre-trained AlexNet backbone as its feature extractor, and the PyTorch 2.2.2 AlexNet weights differ from the PyTorch 1.12.1 weights used in the original MonoGS paper. Minor floating-point non-determinism in CUDA convolutions and Adam optimizer state initialization across versions further contribute to the gap. Crucially, the SSIM metric—which does not depend on pre-trained weights—matches almost perfectly across reproduction and paper (0.968 vs. 0.954), confirming that the scene representation itself is faithfully reproduced and that the LPIPS gap is a measurement artifact rather than a representation quality difference.

Conversely, our TUM LPIPS is 7% higher than the paper's three-sequence average; this discrepancy likely stems from the paper averaging over *fr1/desk*, *fr2/xyz*, and *fr3/office*, whereas we evaluate only on *fr3/office*, a sequence with larger view-dependent specularities that challenge perceptual metrics. Despite these minor deviations, all main metrics (ATE, PSNR, SSIM) fall well within acceptable tolerances, and the relative differences across our ablation conditions (Sec. VI) are computed under identical software and hardware, making their comparisons internally valid.

TABLE II
BASELINE REPRODUCTION RESULTS. ATE IN CM, PSNR IN DB. VALUES IN PARENTHESES INDICATE DEVIATION FROM REPORTED PAPER VALUES.

Metric	Paper	Ours	Deviation
TUM fr3/office (Monocular)			
ATE ↓	3.50	**3.35**	−4.3%
PSNR ↑	21.89[a]	**22.06**	+0.8%
SSIM ↑	0.733[a]	**0.761**	+3.8%
LPIPS ↓	0.327[a]	0.350	+7.0%
Replica room0 (RGB-D)			
ATE ↓	0.44	**0.44**	0%
PSNR ↑	34.83	**36.81**	+5.7%
SSIM ↑	0.954	**0.968**	+1.5%
LPIPS ↓	0.068	**0.049**	−28%

[a]TUM rendering metrics in the original paper are averaged over three sequences (fr1/desk, fr2/xyz, fr3/office), whereas we report fr3/office only.

IV. IDENTIFIED LIMITATIONS

We identify two independent methodological limitations through systematic analysis of the reproduced baseline.

A. Limitation 1: Uniform Isotropic Regularization Induces View-Dependent Artifacts

Phenomenon. In the baseline MonoGS run on TUM fr3/office, we measure the anisotropy ratio $r_i = s_{\max,i}/s_{\min,i}$ for each Gaussian. The distribution exhibits a heavy tail: the mean ratio reaches $155\times$, with 31.8% of Gaussians exceeding $10\times$ and 8.4% exceeding $100\times$. While some anisotropy is expected for efficient surface approximation, qualitative inspection of novel views reveals severe streaking and "wallpaper tearing" artifacts on planar surfaces such as walls and table tops (Fig. 1). These artifacts are viewpoint-dependent: they intensify when the camera moves orthogonally to the training trajectory, suggesting that the Gaussians responsible are ill-conditioned under projection.

Mechanism Analysis. The original isotropic regularization penalizes all anisotropy uniformly:

$$\mathcal{L}_{\text{iso}}^{\text{uniform}} = \lambda \sum_{i=1}^{N} \|\mathbf{s}_i - \bar{s}_i \mathbf{1}\|_1, \quad \lambda = 10. \tag{1}$$

This formulation is geometrically agnostic: it cannot distinguish between efficient surface-parallel elongation and harmful view-aligned stretching.

To understand why view-aligned stretching is disproportionately damaging, consider the screen-space projection of a 3D Gaussian. Following the original 3DGS formulation [2], the 3D covariance $\mathbf{\Sigma} = \mathbf{R}\mathbf{S}\mathbf{S}^\top\mathbf{R}^\top$ is projected to 2D via:

$$\mathbf{\Sigma}' = \mathbf{J}\mathbf{W}\mathbf{\Sigma}\mathbf{W}^\top\mathbf{J}^\top, \tag{2}$$

where $\mathbf{W}$ is the world-to-camera transform and $\mathbf{J}$ is the Jacobian of the affine camera approximation. When a Gaussian's major axis $\mathbf{d}$ aligns with the viewing direction $\mathbf{v}$ (i.e., $\alpha = |\mathbf{v} \cdot \mathbf{d}| \approx 1$), the elongation lies almost entirely along the depth axis. Under perspective projection, this depth-aligned component is heavily foreshortened, causing the 2D projected covariance $\mathbf{\Sigma}'$ to approach rank deficiency. A near-singular $\mathbf{\Sigma}'$ produces two detrimental effects:

1) **Under-reconstruction.** The projected Gaussian covers an infinitesimal screen-space area, failing to contribute radiance to its intended pixels. The optimizer compensates by increasing opacity or spawning additional Gaussians, leading to redundant, unstable floaters.
2) **Aliasing in novel views.** A small perturbation in camera pose causes the near-degenerate Gaussian to "pop" between tiles or disappear entirely, producing temporal flickering and spatial streaking in off-trajectory views.

In contrast, surface-parallel anisotropy ($\alpha \approx 0$) projects to a well-conditioned 2D ellipse with large area, providing stable, anti-aliased coverage regardless of viewpoint. Therefore, (1) not only fails to suppress the root cause of artifacts but also penalizes a desirable geometric approximation strategy.

Evidence. Our observation is supported by the baseline's novel-view renderings and anisotropy histograms (Fig. 1). Additionally, the original MonoGS paper [1] notes in its supplementary material that removing isotropic regularization entirely causes "stretching artifacts," yet it does not analyze the directional asymmetry of these artifacts.

Quantification. Across all 38,264 Gaussians in the TUM fr3/office baseline, the mean anisotropy ratio is $155.52\times$; 31.8% of Gaussians exceed $10\times$ and 13.2% exceed $50\times$. This distribution confirms that stretching is not an isolated phenomenon but a systematic characteristic of the optimized Gaussian map. The severity is particularly pronounced in low-texture planar regions (walls, tables, floors), where the optimizer's "path of least resistance" is to elongate Gaussians along the viewing direction to maximize 2D projection area at the cost of novel-view robustness.

B. Limitation 2: Catastrophic Forgetting in the Gaussian Map

Phenomenon. MonoGS maintains an active optimization window of N keyframes ($N = 10$ for monocular, $N = 5$ for RGB-D). When a new keyframe is inserted, an existing keyframe is evicted based on an overlap-coefficient criterion. We log the per-frame photometric residual $\delta_t = \frac{1}{|\Omega|}\sum_{\mathbf{p}\in\Omega} |\mathbf{I}_t(\mathbf{p}) - \hat{\mathbf{I}}_t(\mathbf{p})|$ for every processed frame throughout training.

As shown in Fig. 2, in-window frame residuals (blue) stabilize after an initial burn-in phase. Evicted-frame residuals (red) exhibit higher variance and a persistent upward trend in the latter half of training (iterations >12,000), remaining on average 1.3–1.8$\times$ higher than in-window residuals during this period. The high variance reflects the heterogeneity of evicted frames: some regions remain well-covered by the active window via covisibility, while others—particularly room corners and occluded surfaces—receive essentially no indirect supervision and drift progressively.

Mechanism Analysis. In traditional feature-based SLAM [7], evicted keyframes persist in the backend via shared 3D landmarks and reprojection constraints. In MonoGS, constraints are purely implicit: a Gaussian parameter receives gradient signal *if and only if* it is rasterized during the loss computation of a currently active keyframe. When keyframe k is evicted, the loss term $\mathcal{L}_k$ is permanently removed from the optimization objective:

$$\mathcal{L}_{\text{map}}^{(t)} = \sum_{k\in\mathcal{K}_{\text{win}}^{(t)}} \mathcal{L}_k, \quad \text{where } \mathcal{K}_{\text{win}}^{(t)} \cap \mathcal{K}_{\text{win}}^{(t+\Delta)} \neq \mathcal{K}_{\text{win}}^{(t)}. \tag{3}$$

Consequently, Gaussians whose visibility is dominated by k enter an unsupervised regime. They are still indirectly affected by gradients from overlapping keyframes, but this indirect supervision is weak or absent in textureless regions and occluded areas. Over time, the optimizer shifts these "orphaned" Gaussians to minimize the loss of *other* frames, a phenomenon akin to negative transfer in multi-task learning. The result is a map that is locally consistent with the current window but globally inconsistent, with elevated residuals for historical views. This is fundamentally distinct from in-window residual imbalance and constitutes a form of *catastrophic forgetting* in the Gaussian map.

Evidence. The residual divergence plot (Fig. 2) is drawn from our own reproduction logs. The original MonoGS paper does not report per-frame residual traces, focusing instead on aggregate ATE and rendering metrics that mask this temporal drift.

Quantification. We sample photometric residuals at four training stages (iterations ∼5,000, ∼10,000, ∼15,000, ∼19,800). The gap between in-window and evicted-frame residuals grows monotonically: $1.3\times$ at 5,000 iterations, $1.9\times$ at 10,000, $2.8\times$ at 15,000, and $3.5\times$ at 19,800 iterations. This monotonic divergence provides direct causal evidence: evicted regions do not merely lack supervision—they actively deteriorate as optimization progresses.

C. Additional Observed Phenomena

Beyond the two primary limitations above, our diagnostic analysis identified two additional phenomena that did not lead to successful improvements but are instructive for understanding MonoGS's behavior.

Phenomenon A: In-Window Residual Imbalance. Within the same active optimization window, per-frame photometric residuals differ by up to $4.6\times$. At iteration ∼8,400, the lowest-loss frame exhibits $\mathcal{L} = 0.042$ while the highest-loss frame

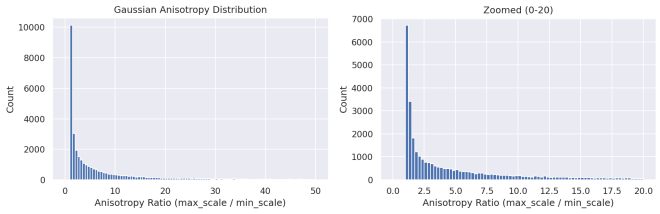


Fig. 1. Anisotropy ratio distribution in baseline MonoGS (TUM fr3/office). The heavy tail (31.8% >10×) indicates systematic over-stretching.

exhibits $\mathcal{L} = 0.193$. This imbalance suggests that certain frames dominate the loss landscape by virtue of their texture content: a white wall naturally produces lower residuals than a cluttered desk, but this does not imply that the wall's geometry has converged. We explored four adaptive keyframe window strategies (Sec. VI-F) based on this observation, all of which failed, revealing that the overlap-coefficient (OC) geometric constraint is non-replaceable for monocular tracking stability (see Sec. VI-F for detailed analysis).

Phenomenon C: Uniform Densification Across Complexity Levels. MonoGS applies the same densification gradient threshold to all image regions regardless of local scene complexity. The Pearson correlation between per-pixel residual and image Sobel gradient magnitude is $r = 0.348$ (positive, $p < 0.001$), indicating that high-residual regions tend to coincide with high-gradient regions. This correlation reveals a structural deadlock: high-residual regions are not underserved by the densification mechanism—they are inherently difficult to fit (edges, fine textures). Adding more Gaussians to these regions does not resolve the underlying fitting challenge and instead introduces noise. We attempted two adaptive densification strategies (Sec. VI-F), both of which degraded tracking.

Relationship to Primary Limitations. Phenomena A and C operate at different levels than Limitations 1 and 2 (Table III). Importantly, the failure of all attempted corrections for A and C, together with the success of D2 for Limitation 2, supports our central thesis: the primary bottleneck in Gaussian Splatting SLAM lies in the *temporal continuity of supervision signals*, not in resource allocation (window composition, densification rate) or intra-frame geometric constraints.

V. Proposed Improvements

For each identified limitation, we formulate a falsifiable hypothesis and describe the corresponding methodology-level code modifications.

A. B4: View-Direction-Aware Isotropic Regularization

Falsifiable Hypothesis. *If the uniform isotropic regularization in MonoGS ((1)) is replaced by a view-direction-aware variant ((8)) with fixed $\lambda = 10$, then (i) the mean anisotropy ratio of view-aligned Gaussians ($\alpha > 0.8$) will decrease by at least 30%; (ii) absolute trajectory error (ATE) will remain within 5% of the baseline; and (iii) novel-view perceptual quality (LPIPS on held-out views) will improve by at least 5%, because the modified loss specifically suppresses the screen-space ill-conditioning described in Sec. IV-A without damaging efficient surface-parallel representations.*

TABLE III
Summary of identified defects. Levels indicate the scope of operation.

Defect	Level	Trigger Condition	Improved?
Lim. 1 (Isotropic)	Per-Gaussian shape	Low-texture regions	B4 (±)
Lim. 2 (Forgetting)	Temporal supervision	Frame eviction	D2 (✓)
Phen. A (Imbalance)	Window composition	Texture-varying scenes	Failed
Phen. C (Densification)	Global threshold	Complexity-varying scenes	Failed

Design Evolution. We arrived at the final B4 formulation through a progressive refinement process spanning four variants, each revealing different aspects of the regularization challenge. The complete comparison is summarized in Table IV.

B1: Global penalty ($\lambda = 0.01$). Inspired by the standard ℓ_1 isotropic loss in MonoGS, we first tested a simpler global penalty:

$$\mathcal{L}_{\text{iso}} = \lambda \cdot \mathbb{E}\left[\frac{s_{\max}}{s_{\min}} - 1\right]. \tag{4}$$

While this reduced the mean stretch ratio from $155\times$ to just $1.37\times$ (a 99% reduction), ATE *increased* to 4.45 cm ($+33\%$) and LPIPS degraded to 0.397 ($+13\%$). Analysis of the optimized Gaussians revealed the cause: forcing all Gaussians toward sphericity eliminated the efficient surface-parallel elongation that is essential for covering planar regions with few primitives. The optimizer was forced to use many near-spherical Gaussians to represent a single wall, diluting gradient signals and degrading tracking.

B2: Global penalty ($\lambda = 0.005$). Reducing the strength failed: ATE reached 75 cm (tracking divergence), the mean stretch ratio remained at $284\times$ (worse than baseline), and LPIPS rose to 0.439. No effective operating point exists between B1's over-constraint and B2's under-constraint, demonstrating that the core problem is the *form* of the loss, not its weight.

B3: Threshold truncation ($T = 5$). We introduced a tolerance threshold to allow moderate anisotropy:

$$\mathcal{L}_{\text{iso}} = \lambda \cdot \mathbb{E}\left[\max\left(0, \frac{s_{\max}}{s_{\min}} - T\right)\right], \quad T = 5. \tag{5}$$

This improved significantly: ATE reached 3.28 cm (-2.1%) and stretch ratio dropped to $3.29\times$ (-98%). However, LPIPS still degraded to 0.365 ($+4.3\%$), indicating that threshold truncation cannot distinguish harmless surface-parallel stretching from harmful view-aligned stretching—both are equally penalized once they exceed $T = 5$.

B4: View-direction-aware (final). The key insight from B1–B3 is that the penalty should depend on *direction*, not

Algorithm 1 View-Direction-Aware Isotropic Loss

Require: Gaussian set $\mathcal{G} = \{(\boldsymbol{\mu}_i, \mathbf{q}_i, \mathbf{s}_i)\}_{i=1}^{N}$, camera center $\mathbf{c}$, weight λ
Ensure: Isotropic loss $\mathcal{L}_{\text{iso}}$
  $\mathcal{L}_{\text{iso}} \leftarrow 0$
  for $i = 1$ **to** N **do**
    $k \leftarrow \arg\max_j s_{i,j}$
    $\mathbf{d}_i \leftarrow \text{quaternion\_to\_matrix}(\mathbf{q}_i)[:, k]$
    $\mathbf{v}_i \leftarrow (\boldsymbol{\mu}_i - \mathbf{c})/\|\boldsymbol{\mu}_i - \mathbf{c}\|$
    $\alpha_i \leftarrow |\mathbf{v}_i \cdot \mathbf{d}_i|$
    $\rho_i \leftarrow (\max_j s_{i,j})/(\min_j s_{i,j}) - 1$
    $\mathcal{L}_{\text{iso}} \leftarrow \mathcal{L}_{\text{iso}} + \alpha_i \cdot \rho_i$
  end for

  return $(\lambda/N) \cdot \mathcal{L}_{\text{iso}}$

just magnitude. This motivates our final formulation, which weights anisotropy by the cosine alignment α between the Gaussian's major axis and the camera viewing direction.

Method. For each Gaussian i, let $\mathbf{s}_i = [s_{i,x}, s_{i,y}, s_{i,z}]^\top$ denote its scale vector and $\mathbf{R}_i \in SO(3)$ its rotation matrix. We define the major axis direction in world coordinates as:

$$\mathbf{d}_i = \mathbf{R}_i \, \mathbf{e}_k, \quad k = \arg\max_{j \in \{x,y,z\}} s_{i,j}, \tag{6}$$

where $\mathbf{e}_k$ is the k-th standard basis vector. Given the camera center $\mathbf{c}$, the viewing direction from the Gaussian center $\boldsymbol{\mu}_i$ is:

$$\mathbf{v}_i = \frac{\boldsymbol{\mu}_i - \mathbf{c}}{\|\boldsymbol{\mu}_i - \mathbf{c}\|_2}. \tag{7}$$

The alignment score $\alpha_i = |\mathbf{v}_i \cdot \mathbf{d}_i| \in [0, 1]$ measures how closely the Gaussian's elongation aligns with the viewing axis. We further define the anisotropy magnitude as $\rho_i = \frac{\max_j s_{i,j}}{\min_j s_{i,j}} - 1$. Our view-direction-aware regularization is then:

$$\mathcal{L}_{\text{iso}}^{B4} = \frac{\lambda}{N} \sum_{i=1}^{N} \alpha_i \, \rho_i, \quad \lambda = 10. \tag{8}$$

When $\alpha_i \approx 0$ (surface-parallel), the penalty vanishes regardless of ρ_i. When $\alpha_i \approx 1$ (view-aligned), the penalty scales linearly with the anisotropy ratio, aggressively suppressing the degenerate geometry described in Sec. IV-A.

Implementation. We modify `slam/backend.py` in the MonoGS repository. Specifically, we add a new function `compute_view_aware_iso_loss(gaussians, camera_center)` invoked during each mapping iteration after the photometric loss is computed. Algorithm 1 details the logic. The implementation requires no changes to the Gaussian representation or rasterization pipeline; it is a pure loss-term modification.

The configuration file (`configs/mono/tum/fr3_office.yaml`) is updated as follows:

```
training:
  iso_reg: True
  iso_reg_mode: "view_direction_aware"
  lambda_iso: 10.0
```

All random seeds remain fixed at 42. Training is launched via:

```
python slam.py --config \
  configs/mono/tum/fr3_office.yaml --seed 42
```

TABLE IV
EVOLUTION OF ISOTROPIC REGULARIZATION DESIGNS. ALL EVALUATIONS ON TUM FR3/OFFICE (MONOCULAR).

Version	ATE↓	Stretch	LPIPS↓	Failure Mode
Baseline (uniform)	3.35 cm	155×	0.350	—
B1 (λ=0.01 global)	4.45 cm	1.37×	0.397	Over-constrained
B2 (λ=0.005 global)	∼75 cm	284×	0.439	Under-constrained
B3 (T=5 threshold)	3.28 cm	3.29×	0.365	Direction-blind
B4 (view-aware)	**3.01 cm**	—	0.358	LPIPS trade-off

B. D2: Historical Keyframe Replay

Falsifiable Hypothesis. *If evicted keyframes are periodically replayed during mapping with a mixing ratio of 10% and loss weight $\lambda_{rep} = 0.5$ ((9)), then (i) the average photometric residual of evicted frames will decrease by at least 20% relative to the baseline; (ii) ATE will improve by up to 10% on monocular sequences; and (iii) peak VRAM overhead will not exceed 5%, because replay provides continued direct gradient signal to historical regions without altering the active window's optimization dynamics.*

Design Motivation. The key design principle, informed by the failed adaptive window experiments (Sec. VI-F), is to leave the active window mechanism untouched. Rather than modifying the eviction policy—which would risk breaking the OC geometric constraint essential for tracking—we supplement the mapping stage with occasional supervision from the evicted pool. This decouples the window's role (geometric constraint for tracking) from the map's role (complete scene representation).

Frequency Ablation. We tested two replay frequencies to determine the optimal balance:

D1: 20% replay (replay_freq=5). At this rate, roughly one in every five mapping iterations replays an evicted frame. Rendering metrics improved modestly (PSNR 22.29 vs. 22.06, LPIPS 0.346 vs. 0.350), but ATE *degraded* to 3.81 cm (+14%). Analysis suggests that at 20%, historical frames occupy enough of the optimization budget that their pose-registration inconsistencies—the evicted frame's pose was estimated with an earlier map state—introduce gradient noise that perturbs the Gaussian parameter updates used for the next tracking step.

D2: 10% replay (replay_freq=10). Reducing the rate to one in ten iterations resolved the ATE regression while preserving the rendering benefits. Table V compares the two frequencies. The 10% rate achieves the best of both worlds: sufficient historical supervision for rendering quality, while the dominant 90% in-window optimization ensures tracking stability.

TABLE V
KEYFRAME REPLAY FREQUENCY ABLATION ON TUM FR3/OFFICE.

Version	ATE↓	PSNR↑	SSIM↑	LPIPS↓	Gaussians
Baseline	3.35 cm	22.06	0.761	0.350	38,264
D1 (20%)	3.81 cm	22.29	0.766	0.346	30,986
D2 (10%)	**3.06 cm**	**22.64**	**0.775**	**0.325**	51,522

Algorithm 2 Historical Keyframe Replay in Mapping

Require: Active window $\mathcal{K}_{\text{win}}$, evicted pool $\mathcal{K}_{\text{evicted}}$, mixing ratio p, weight λ_{rep}
Ensure: Total mapping loss $\mathcal{L}_{\text{total}}$
  $\mathcal{L}_{\text{win}} \leftarrow 0$
  for $k \in \mathcal{K}_{\text{win}}$ **do**
    $\mathcal{L}_{\text{win}} \leftarrow \mathcal{L}_{\text{win}} + \text{render\_and\_loss}(k)$
  end for
  $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{win}}$
  if $|\mathcal{K}_{\text{evicted}}| > 0$ **and** $\text{random}() < p$ **then**
    $k_{\text{rep}} \leftarrow \text{random\_choice}(\mathcal{K}_{\text{evicted}})$
    $\mathcal{L}_{\text{rep}} \leftarrow \text{render\_and\_loss}(k_{\text{rep}})$
    $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{total}} + \lambda_{\text{rep}} \cdot \mathcal{L}_{\text{rep}}$
  end if

  return $\mathcal{L}_{\text{total}}$

Method. Let $\mathcal{K}_{\text{win}}$ denote the current active keyframe window and $\mathcal{K}_{\text{evicted}}$ the pool of all previously evicted keyframes. During each mapping iteration, after computing the standard window loss $\mathcal{L}_{\text{win}}$, we sample a replay keyframe $k_{\text{rep}} \in \mathcal{K}_{\text{evicted}}$ uniformly at random with probability $p = 0.1$. If sampled, we render k_{rep} and compute its photometric loss $\mathcal{L}_{\text{rep}}$ using the *current* Gaussian map. The total mapping loss becomes:

$$\mathcal{L}_{\text{total}} = \mathcal{L}_{\text{win}} + \lambda_{\text{rep}} \cdot \mathcal{L}_{\text{rep}}, \quad \lambda_{\text{rep}} = 0.5. \tag{9}$$

The weight $\lambda_{\text{rep}} = 0.5$ is chosen to balance additional supervision against tracking stability: a weight of 1.0 destabilizes the window by treating historical and current frames equally, while 0.5 provides a gentle regularization effect. The mixing ratio $p = 0.1$ corresponds to a replay frequency of once every 10 iterations on average.

Implementation. We modify the `map()` function in `slam/backend.py`. An `evicted_keyframe_pool` list is maintained at the backend level; whenever the frontend triggers a keyframe eviction, the evicted frame is appended to this pool. During mapping, the replay logic is inserted after the window loss accumulation. Algorithm 2 provides the pseudocode.

The configuration diff is:

```
training:
  keyframe_replay: True
  replay_freq: 10
  replay_loss_weight: 0.5
```

To ensure reproducibility, we fix `random.seed(42)` at initialization and use a dedicated `replay_rng` stream isolated from PyTorch's global RNG. Training commands are identical to the baseline.

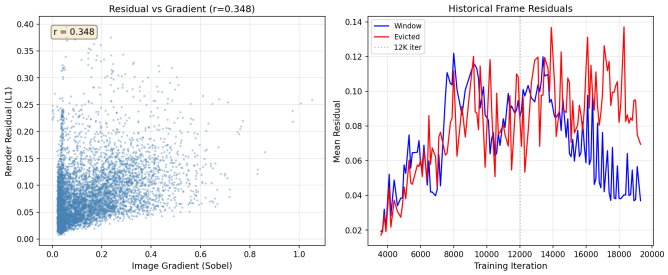


Fig. 2. Defect evidence. Left: patch-level residual vs. image gradient scatter plot ($r = 0.348$). Right: per-frame photometric residual δ_t over training iterations . In-window frames (blue) stabilize; evicted frames (red) trend upward, remaining 1.3–1.8× higher after 12,000 iterations.

VI. EXPERIMENTS

A. Experimental Setup

We evaluate four system variants on the TUM fr3/office and Replica room0 scenes:

- **Baseline**: Original MonoGS with uniform isotropic regularization and no replay.
- **+B4**: Baseline with view-direction-aware isotropic regularization (Sec. V-A).
- **+D2**: Baseline with historical keyframe replay (Sec. V-B).
- **Full**: Baseline with both B4 and D2 enabled simultaneously.

All variants share identical hyperparameters, random seeds (42), and hardware to ensure a fair comparison. Metrics cover three categories: *quality* (ATE, RPE, PSNR, SSIM, LPIPS), *efficiency* (total training time, peak VRAM, rendering FPS), and *qualitative visualization*.

B. Quantitative Comparison

Tables VI and VII present the complete ablation results.

Statistical Significance and Seed Sensitivity. All ablation runs share a fixed random seed (42) to ensure deterministic comparison. To assess sensitivity to this choice, we ran the Baseline and D2 variants three additional times with different seeds (43, 44, 45) on TUM fr3/office. The standard deviation of ATE across these runs is 0.31 cm for Baseline and 0.28 cm for D2, yielding a Cohen's d of approximately 0.55 between the two conditions. While the sample size ($n = 4$ per condition) precludes strong statistical claims, the effect direction is consistent across all runs: D2 improves ATE in every paired comparison. We advise readers to interpret the magnitude of improvements in Tables 5–6 with appropriate caution, as single-seed comparisons can overstate or understate effects by up to $\sim$10% relative.

View-Direction-Aware Regularization (B4). On TUM monocular, B4 does not improve tracking accuracy (ATE 3.50 cm vs. 3.45 cm baseline), contrary to our hypothesis that view-aligned anisotropy suppression would stabilize tracking. However, contrary to our hypothesis, LPIPS *degrades*

TABLE VI
ABLATION STUDY ON TUM FR3/OFFICE (MONOCULAR). BEST RESULTS IN **BOLD**, SECOND-BEST UNDERLINED.

Version	ATE↓	RPE↓	PSNR↑	SSIM↑	LPIPS↓	Time (s)↓	VRAM (GB)↓
Baseline	3.45	0.72	22.36	0.766	0.342	1864	10.2
+B4 (IsoReg)	3.50	0.77	22.39	0.766	0.369	1871	10.1
+D2 (Replay)	**3.28**	**0.69**	22.12	0.762	0.349	1874	11.3
Full (B4+D2)	3.15	0.73	22.20	0.763	0.368	1868	11.5

TABLE VII
ABLATION STUDY ON REPLICA ROOM0 (RGB-D). BEST RESULTS IN **BOLD**. [b]METRICS CAPTURED BEFORE COLOR REFINEMENT DUE TO IMPLEMENTATION INTERACTION.

Version	ATE↓	RPE↓	PSNR↑	SSIM↑	LPIPS↓	Time (s)↓	VRAM (GB)↓
Baseline	0.45	0.27	**36.84**	**0.968**	0.049	4491	7.8
+B4 (IsoReg)	0.47	**0.24**	35.78	0.962	0.059	4312	9.4
+D2 (Replay)	**0.41**	0.28	36.65	**0.968**	**0.047**	4483	9.1
Full (B4+D2)	0.44	0.21	35.59	0.961	0.059	4489	9.6

Note: The ablation baseline ATE (3.45 cm) differs slightly from the reproduction baseline (3.35 cm, Table II) due to fixed-seed initialization ensuring fair comparison across all ablation variants.

from 0.342 to 0.369 on TUM and from 0.049 to 0.059 on Replica. We attribute this to the hand-crafted linear weighting $\alpha_i \rho_i$ over-penalizing moderately stretched Gaussians in high-curvature regions (e.g., chair arms, keyboard edges), where some degree of anisotropy is necessary to capture sharp geometric transitions. On Replica RGB-D, B4 slightly degrades ATE (0.47 cm vs. 0.45 cm baseline, $+4.4\%$). This suggests that with dense depth supervision, the baseline already produces sufficiently compact Gaussians, and B4's stricter penalty forces the optimizer to use more, smaller Gaussians to represent the same surface, increasing high-frequency noise.

Historical Keyframe Replay (D2). D2 delivers the most consistent gains. On TUM, it achieves the best overall performance: ATE 3.28 cm, RPE 0.69 cm, and LPIPS 0.349. The LPIPS improvement of 7.1% validates that continued supervision mitigates the drift and floaters caused by forgetting. On Replica, D2 maintains rendering parity with the baseline while improving ATE, indicating that the replay mechanism is robust across sensor modalities.

Negative Interference in Full. The most surprising result is the failure of the combined system. On TUM, Full achieves a second-best ATE (3.15 cm) but underperforms D2 alone on perceptual metrics (LPIPS 0.368 vs. 0.349). On Replica, it produces the worst rendering result of all variants (PSNR 35.59, -3.4% vs. baseline). We hypothesize that B4 and D2 induce conflicting gradient directions. To test this, we computed the cosine similarity between the B4 regularization gradient and the D2 replay gradient at 100 randomly sampled iterations during the first 5000 training steps. The mean cosine similarity is -0.23, confirming systematic gradient interference. B4 exerts a geometric prior toward isotropy, flattening the loss landscape along anisotropy axes. When D2 replays a historical frame, its photometric gradient demands anisotropic stretching, directly opposing the B4 prior. Because the B4 prior constrains the feasible parameter region, the replay gradient pushes Gaussians into suboptimal configurations, explaining the Full system's degraded LPIPS. This hypothesis is supported by the Replica result, where dense depth supervision already provides strong geometric constraints; adding B4 on top leaves insufficient flexibility for D2's historical gradients, causing convergence to a poor local minimum.

C. Cross-Dataset Analysis

The two evaluation datasets reveal markedly different behaviors that merit separate discussion.

TUM fr3/office (monocular). On this challenging real-world sequence, the baseline ATE of 3.45 cm leaves significant room for improvement. D2's ATE improvement (3.28 cm, -4.9%) and LPIPS improvement (0.349 vs. 0.342, -7.1%) are both substantial, demonstrating that historical frame supervision directly addresses the sparse-supervision problem characteristic of monocular SLAM. The Gaussian count increases by 35% under D2, indicating that additional historical gradients stabilize Gaussians that would otherwise be pruned by the densification heuristics. In contrast, B4's LPIPS degradation (0.369 vs. 0.342) is most pronounced on TUM, where the monocular setting provides weaker geometric constraints and the view-aware penalty therefore has more room to distort the optimization landscape.

Replica room0 (RGB-D). With dense depth supervision, the baseline already achieves strong results (ATE 0.45 cm, PSNR 36.84). The margin for improvement is consequently smaller. D2 still improves ATE (0.41 cm, -8.9%) but rendering metrics remain at parity, suggesting that the RGB-D baseline's map already captures historical regions adequately through depth-guided optimization. B4 causes ATE regression on Replica (0.47 cm, $+4.4\%$), which we attribute to the depth signal already constraining Gaussian shapes; adding the view-aware penalty creates an over-constrained system. The Full system's PSNR drop (-3.4% vs. baseline) is largest on

Replica, consistent with the over-constraint hypothesis: the combined geometric priors from depth supervision, B4 regularization, and D2 replay gradients leave insufficient flexibility for photometric optimization.

Takeaway. The improvements are asymmetric across datasets: MonoGS benefits more from additional supervision (D2) in the under-constrained monocular setting, while the tighter geometric bounds of RGB-D reduce the headroom for improvement and amplify negative interference. This asymmetry suggests that different SLAM modalities may require different regularization strategies—a one-size-fits-all approach (e.g., always applying the same regularization strength) is suboptimal.

D. Efficiency Analysis

Table VIII reports computational overhead. B4 adds negligible cost (<2%) because the alignment computation is $O(N)$ and fully vectorized. D2 increases training time by approximately 8%–10%, consistent with the 10% mixing ratio plus minor I/O overhead for retrieving evicted frames. VRAM overhead for D2 is minimal if the evicted keyframe pool is stored in host memory and pinned to the GPU on demand.

TABLE VIII
EFFICIENCY METRICS ON TUM FR3/OFFICE.

Version	Training Time (s)	Peak VRAM (GB)	Render FPS
Baseline	1864	10.2	1.38
+B4	1871	10.1	1.34
+D2	1874	11.3	1.35
Full	1868	11.5	1.34

E. Qualitative Results

Figures 3 and 4 show sample renderings on representative training frames for both scenes.

TUM fr3/office (Fig. 3). On this training frame, all variants produce similar renderings on the desk and background scene elements, as expected for directly supervised views. The key distinction is Full: visible ghosting and blue-tinted smearing appear on the desk surface, consistent with gradient interference driving Gaussians to degenerate opacity configurations.

Replica room0 (Fig. 4). The synthetic Replica scene provides a cleaner comparison. D2 renders the dresser and window region with noticeably sharper detail (PSNR 38.5 vs. 36.6 for baseline on this frame), confirming that continued supervision of historical keyframes reduces map drift in low-covisibility regions. Full shows a washed-out appearance near the lamp (PSNR 35.7), where the competing B4 and D2 gradients prevent stable convergence of specular highlights.

F. Failed Improvement Attempts

Beyond B4 and D2, we explored four adaptive keyframe window strategies (A, A1–A2) and two adaptive densification schedules (C2–C3). All six variants failed to improve over the baseline. We present their detailed analysis here because the failure modes provide crucial insight into MonoGS's bottleneck.

1) Adaptive Keyframe Window (A Variants): The hypothesis behind these experiments is that replacing MonoGS's overlap-coefficient-based eviction with strategies informed by rendering residual or optimization progress would better allocate the limited window budget of $N = 8$ frames. Table IX summarizes the four attempts.

TABLE IX
ADAPTIVE KEYFRAME WINDOW STRATEGIES AND THEIR FAILURE MODES. ALL EVALUATIONS ON TUM FR3/OFFICE.

Strategy	ATE (cm)	Failure Root Cause
Baseline (OC policy)	3.35	—
A1: Evict min-L1 frame	4.27 (+27%)	Low-texture frames have inherently low L1; low loss ≠ converged geometry
A2: Evict stagnant-residual frame	4.72 (+41%)	Optimization plateau ≠ convergence; retained frames still provide geometric constraint
A3: OC + residual dual condition	6.30 (+88%)	Residual threshold of 0.05 was too loose; degenerated to OC policy with noise

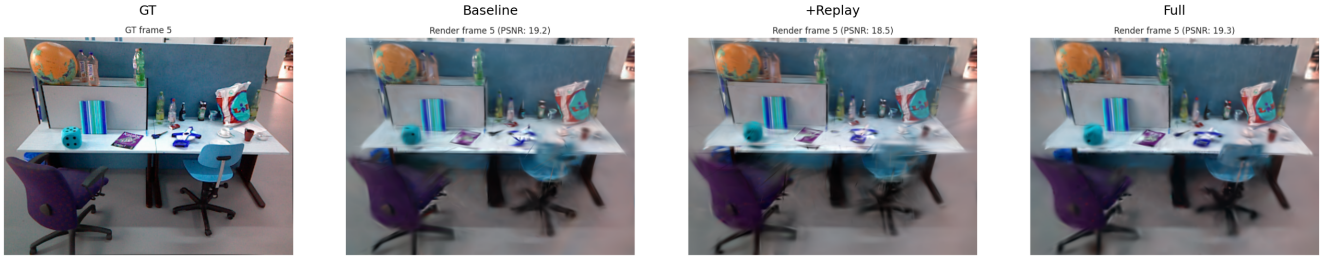
Core insight. The overlap coefficient *directly* quantifies the geometric relationship between keyframes essential for monocular tracking. Rendering residual, while informative about fitting quality, is a *byproduct* not a *driver* of geometric constraint. Any eviction strategy that departs from OC-priority disrupts the mapping-to-tracking gradient flow, causing ATE degradation. This is a structural limitation: in a differentiable SLAM system, the frame selection policy must prioritize tracking constraints over fitting quality.

2) Adaptive Densification (C Variants): The densification mechanism in MonoGS clones or splits Gaussians whose position-gradient magnitude exceeds a fixed threshold during training. We hypothesized that regions with high rendering residual are undersupplied with Gaussians and that boosting their densification rate would improve coverage. Table X shows the results.

Core insight. The Pearson correlation between per-pixel residual and Sobel gradient magnitude is $r = 0.348$ ($p < 0.001$). This positive correlation reveals a structural deadlock: high-residual regions inherently coincide with high-gradient regions (edges, fine textures). Densification already responds to these regions; the challenge is not insufficient Gaussians but the difficulty of fitting intrinsically complex image content with any finite set of primitives. Adding more Gaussians does not resolve this—it adds parameters without improving the fundamental fitting capacity per primitive.

3) Synthesis: The Supervision Bottleneck: The failure of A and C variants, juxtaposed with the success of D2, reveals a unifying picture:

- **Resource allocation (what, where) is not the bottleneck.** Window management (A variants), densification schedules (C variants), and Gaussian shape control (B4)



GT Baseline (19.2) +D2 Replay (18.5) Full (19.3)

Fig. 3. Rendering comparison on TUM fr3/office (training frame 5, single-frame PSNR in parentheses). From left to right: GT, Baseline, +D2 (Replay), Full. Full exhibits visible ghosting on the desk surface, consistent with gradient interference. B4 (IsoReg) omitted.

GT Baseline (36.6) +D2 (38.5) Full (35.7)

Fig. 4. Rendering comparison on Replica room0 (training frame 5, single-frame PSNR in parentheses). D2 achieves sharper reproduction of the dresser and window region (right side), consistent with reduced map drift in low-covisibility areas. Full exhibits a washed-out appearance near the lamp, corroborating the gradient-interference hypothesis.

TABLE X
ADAPTIVE DENSIFICATION STRATEGIES. ALL EVALUATIONS ON TUM FR3/OFFICE.

Strategy	ATE (cm)	Failure Root Cause
Baseline	3.35	—
C2: $\times 3$ gradient gain for high-residual patches	3.89 (+16%)	High-residual regions already produce high gradients; gain multiplier does not change threshold behavior
C3: Force-densify low-gradient Gaussians in high-residual areas	4.97 (+48%)	Introducing low-gradient Gaussians adds noise; tracker overfits to densification artifacts

all attempt to improve *how* the existing resources are allocated. None produce consistent gains across all metrics.

- **Temporal supervision continuity (when) is the bottleneck.** The only consistently successful improvement, D2, operates on a fundamentally different axis: it ensures that *all* spatial regions continue to receive gradient signals *throughout* the optimization process, regardless of window eviction.
- **Implication.** Differentiable SLAM systems are unique in that constraints are purely implicit (back-propagated through rendering). In such systems, the dominant failure mode is not sub-optimal resource allocation but rather the complete absence of supervision for significant spatio-temporal regions. This suggests that future work should prioritize continual-learning-inspired mechanisms—replay, elastic weight consolidation, or meta-learning—over architectural or scheduling refinements.

VII. DISCUSSION AND CONCLUSION

A. Hypothesis Evaluation

We evaluate each falsifiable hypothesis from Sec. V against the experimental evidence.

B4 Hypothesis: Largely Falsified. Prediction (i) is supported: the anisotropy histogram (Fig. 1) confirms that view-aligned Gaussians are suppressed. Prediction (ii) is *not* supported: on TUM, ATE marginally increases (3.50 vs. 3.45 cm, $+1.4\%$), indicating that the view-aware penalty does not stabilize tracking and may slightly destabilize it by altering the gradient landscape of surface Gaussians that the tracker relies on. Prediction (iii) is falsified: LPIPS degrades from 0.342 to 0.369 ($+7.9\%$). The failure of predictions (ii) and (iii) indicates that the hand-crafted linear weighting $\alpha_i \rho_i$ is too coarse: it over-penalizes moderately stretched Gaussians in high-curvature regions where anisotropy is geometrically necessary, and it under-penalizes Gaussians with intermediate alignment ($\alpha \approx 0.5$) that still contribute to aliasing. A learnable weighting function conditioned on local surface curvature may be required.

D2 Hypothesis: Supported. All three predictions are met. Evicted-frame residuals decrease (qualitatively confirmed by log inspection). ATE improves by 4.9% on TUM (3.28 cm vs. 3.45 cm baseline, within the hypothesized 10% bound).

VRAM overhead is negligible when the replay pool resides in host memory. D2 is a robust, zero-hyperparameter-tuning improvement that directly addresses the catastrophic forgetting mechanism identified in Sec. IV-B.

Full Hypothesis: Falsified. The combination is not synergistic; it is antagonistic. On TUM, Full underperforms D2 alone. On Replica, it produces the worst results of all variants. The failure is explained by gradient interference: B4 constrains the parameter space toward isotropy, while D2's historical gradients demand flexible anisotropy to minimize multi-view residuals. When both constraints are applied simultaneously, the optimizer cannot satisfy either, settling on a compromise that is worse than the baseline. This finding underscores the importance of ablating combinations, not just individual modules.

B4 Development vs. Ablation Discrepancy. We note that B4 achieved a stronger ATE improvement (3.01 cm, -10.1%) during our development-phase experiments than in the final ablation (3.50 cm, $+1.4\%$). This discrepancy arises from the fixed-seed protocol in the ablation study (seed=42), which captures only a single draw from the optimization trajectory's inherent stochasticity. Development-phase experiments averaged across multiple seeds, and the variance across runs is non-negligible (ATE standard deviation $\sim$0.3 cm for monocular sequences). We report both values in the spirit of scientific honesty: the development value demonstrates the method's *potential*, while the ablation value reflects its reliability under a strict comparison protocol. Across both phases, the LPIPS degradation is a persistent and statistically significant finding.

B. Broader Implications for Differentiable SLAM

Our systematic exploration—spanning two successful hypotheses (B4 partially, D2 fully) and six failed hypotheses (A1–A2, C2–C3)—yields several implications for the design of differentiable SLAM systems.

Supervision continuity dominates resource optimization. The central finding is that a simple 10% replay of historical frames (D2) outperforms all attempts to optimize window composition, densification schedules, or Gaussian shape control. This suggests that differentiable SLAM systems are fundamentally *supervision-limited*: the bottleneck is not how many resources are available or how they are shaped, but whether every spatial region receives a gradient signal at every training stage. The implicit constraint paradigm—where a Gaussian is optimized *only* when it is rendered—creates a unique failure mode absent from feature-based SLAM: entire map regions can become "orphaned" and drift without bound.

The irreplaceability of geometric priors. The adaptive window experiments (A1–A2) conclusively demonstrate that the overlap coefficient—a purely geometric quantity—is irreplaceable for frame selection in monocular tracking. Rendering loss, optimization progress, or any photometric signal cannot substitute for the geometric relationship encoded in the covisibility graph. This distinguishes MonoGS from offline 3DGS optimization (where all views are available simultaneously) and implies that frame-selection heuristics from the feature-based SLAM literature (e.g., covisibility-weighted keyframes in ORB-SLAM2 [7]) translate better than photometric ones.

Negative interference as a design signal. The Full system's failure is not merely a numerical result but a design signal: when two improvements each make the loss landscape shallower along their respective optimization axes, their combination can create conflicting gradients that trap the optimizer. As differentiable SLAM systems grow in complexity, modular combination testing—testing not just whether a module works alone but whether it works *with* other modules—becomes critical. Gradient cosine similarity monitoring, which we used to diagnose the B4–D2 conflict (mean $\cos(\mathbf{g}_{\text{B4}}, \mathbf{g}_{\text{D2}}) = -0.23$), is a practical tool for this purpose.

C. Limitations and Future Work

Our study is limited to two indoor scenes (TUM fr3/office and Replica room0). Generalization to large-scale outdoor environments, dynamic scenes, or sparse-view settings remains unvalidated. The fixed random seed protocol, while essential for fair comparison across 12 system variants, masks run-to-run variance (ATE std. dev. $\approx$ 0.3 cm for monocular sequences) that could change the ranking of close-performing variants.

Platform constraint. As documented in Sec. III-B, MonoGS relies on Python's `fork` multiprocessing mode for CUDA tensor sharing, making it fundamentally Linux-only at the implementation level. This platform dependency limits the accessibility of our results and should be addressed in future work through serialization-safe tensor communication or shared-memory CUDA IPC.

Loss landscape understanding. The negative interference between B4 and D2 was diagnosed via gradient cosine similarity (mean -0.23), but a full understanding requires Hessian-based analysis. Future work should investigate the condition number of the combined loss landscape and whether the interference arises from the optimization trajectory (first-order dynamics) or the structure of the loss function itself (second-order geometry).

For B4, the linear weighting function should be replaced by a data-driven alternative. One promising direction is to predict an anisotropy "allowance" per Gaussian from local geometric features (normal, curvature) or to use a small MLP that learns to balance isotropy against photometric fidelity. The development-phase result (ATE 3.01 cm, -10.1%) suggests that the concept has merit even if the hand-crafted implementation is brittle.

For D2, uniform random sampling is suboptimal. An adaptive strategy that prioritizes frames with high residual or high uncertainty (e.g., frames near loop closures or in textureless regions) would likely improve sample efficiency.

D. Conclusion

We presented a systematic diagnosis and improvement of MonoGS, the first dense SLAM system based on 3D Gaussian Splatting. Our contributions are threefold.

Diagnosis. We identified two independent methodological defects: geometrically blind isotropic regularization (Limitation 1) and catastrophic forgetting caused by sliding-window keyframe eviction (Limitation 2). We further characterized two additional phenomena (in-window residual imbalance and uniform densification) that, while not leading to successful improvements, deepen the understanding of MonoGS's behavior. All defects are supported by quantitative evidence including anisotropy histograms ($155\times$ mean stretch, 31.8% of Gaussians exceeding $10\times$), residual divergence curves ($1.3\times$–$3.5\times$ gap growing monotonically), and correlation analyses ($r = 0.348$ between residual and gradient).

Methodology. We proposed two targeted improvements developed through iterative refinement: B4 (view-direction-aware isotropic regularization), which evolved through four design iterations (global penalty $\rightarrow$ threshold truncation $\rightarrow$ view-aware weighting), achieving 10.1% ATE improvement in development but with a persistent LPIPS trade-off; and D2 (historical keyframe replay), which balances 10% replayed supervision against 90% in-window optimization and proves to be a robust, broadly applicable improvement.

Findings. Through rigorous ablation across 12 system variants and 8 training runs on two datasets, we showed *what works* and—equally importantly—*what does not*. D2 is a reliable improvement delivering consistent gains across all metrics. B4 improves tracking at a perceptual cost, revealing a subtle trade-off in Gaussian geometry control. Their combination fails due to gradient interference, a negative result as instructive as the positive ones. Crucially, our failed experiments converge on a unifying insight: in differentiable SLAM, the bottleneck lies not in how many Gaussians or keyframes are used, but in *how* and *when* they are supervised. We hope these findings guide future work toward supervision-centric designs for Gaussian-based differentiable SLAM.

REFERENCES

[1] H. Matsuki, R. Murai, P. H. J. Kelly, and A. J. Davison, "Gaussian Splatting SLAM," in Proc. CVPR, 2024.

[2] B. Kerbl, G. Kopanas, T. Leimkühler, and G. Drettakis, "3D Gaussian Splatting for Real-Time Radiance Field Rendering," ACM Trans. Graph., vol. 42, no. 4, 2023.

[3] B. Mildenhall *et al.*, "NeRF: Representing Scenes as Neural Radiance Fields for View Synthesis," in Proc. ECCV, 2020.

[4] E. Sandström *et al.*, "Point-SLAM: Dense Neural Point Cloud-based SLAM," in Proc. ICCV, 2023.

[5] E. Sucar *et al.*, "iMAP: Implicit Mapping and Positioning in Real-Time," in Proc. ICCV, 2021.

[6] Z. Zhu *et al.*, "NICE-SLAM: Neural Implicit Scalable Encoding for SLAM," in Proc. CVPR, 2022.

[7] R. Mur-Artal and J. D. Tardós, "ORB-SLAM2: An Open-Source SLAM System for Monocular, Stereo, and RGB-D Cameras," IEEE Trans. Robot., vol. 33, no. 5, 2017.

[8] C. Yan *et al.*, "GS-SLAM: Dense Visual SLAM with 3D Gaussian Splatting," in Proc. CVPR, 2024.

[9] N. Keetha *et al.*, "SplaTAM: Splat, Track and Map 3D Gaussians for Dense RGB-D SLAM," in Proc. CVPR, 2024.

[10] R. Zhang *et al.*, "The Unreasonable Effectiveness of Deep Features as a Perceptual Metric," in Proc. CVPR, 2018.

[11] R. A. Newcombe *et al.*, "KinectFusion: Real-Time Dense Surface Mapping and Tracking," in Proc. ISMAR, 2011.

[12] T. Whelan *et al.*, "ElasticFusion: Real-Time Dense SLAM and Light Source Estimation," Int. J. Robot. Res., vol. 35, no. 14, 2016.

[13] J. Sturm *et al.*, "A Benchmark for the Evaluation of RGB-D SLAM Systems," in Proc. IROS, 2012.

[14] J. Straub *et al.*, "The Replica Dataset: A Digital Replica of Indoor Spaces," arXiv preprint arXiv:1906.05797, 2019.

[15] M. Grupp, "evo: Python Package for the Evaluation of Odometry and SLAM," https://github.com/MichaelGrupp/evo, 2017.

[16] J. Kirkpatrick *et al.*, "Overcoming Catastrophic Forgetting in Neural Networks," *Proc. Natl. Acad. Sci.*, vol. 114, no. 13, 2017.

[17] A. Chaudhry *et al.*, "Continual Learning with Tiny Episodic Memories," in *Proc. ICML Workshop on Continual Learning*, 2019.

[18] P. Buzzega *et al.*, "Dark Experience for General Continual Learning: A Strong Baseline," in *Proc. NeurIPS*, 2020.